AI-POWERED STUDENT PERFORMANCE PREDICTION & STUDY RECOMMENDATION SYSTEM

Internship Project Report — Python, Data Visualization, Machine Learning and AI

Note: This is an educational demonstration using a small synthetic dataset. It is not a validated tool for real academic decisions.

Executive Summary

The project demonstrates how Python can be used to explore student learning indicators, visualize patterns, train a machine-learning classifier and present simple study recommendations. A user can enter study hours, attendance, absences, assignment score and previous test score. The application estimates a pass/at-risk category and displays suggestions based on the entered values.

1. Introduction

Student performance can be influenced by study habits, attendance, assignment completion and prior assessment results. This project uses a small illustrative dataset to demonstrate a basic data-science workflow: data loading, visualization, model training, evaluation and prediction.

2. Objectives

1. Load and inspect structured data using Pandas. 2. Visualize learning indicators using Matplotlib. 3. Train a Random Forest classification model using Scikit-learn. 4. Estimate a category for new sample inputs. 5. Provide simple study recommendations. 6. Present the results through an interactive Streamlit interface.

3. Tools and Technologies

Python is the programming language. Pandas handles tabular data. Matplotlib creates charts. Scikit-learn trains and evaluates the classifier. Streamlit provides the interactive web interface.

4. Dataset Description

The included CSV contains 30 synthetic sample records. Features are study hours per day, number of absences, attendance percentage, assignment score and previous test score. The target column is passed (1 = pass, 0 = not pass). These values are invented for demonstration and do not represent actual students.

5. Methodology

The application loads the CSV, selects five input features and the target label, then divides the data into training and test subsets. A Random Forest classifier is trained on the training subset. Accuracy is calculated on the test subset. The dashboard displays summary metrics and charts. When a user enters new values, the model returns a predicted class and estimated probability.

6. System Functionalities

Dashboard: displays sample size, sample pass rate, a scatter plot, a class distribution chart and the data table. Prediction: accepts five user inputs and estimates pass/at-risk status. Recommendations: checks study hours, attendance, assignments, previous score and absences to generate practical suggestions. Model insight:

displays feature importance from the trained model.

7. Implementation Strategy

Install Python and the dependencies listed in requirements.txt. Keep app.py and sample_student_data.csv in the same folder. Run `streamlit run app.py` in the terminal. Open the local address shown by Streamlit in a browser. Test the dashboard and prediction tab with different values.

8. Testing and Evaluation

The application can be tested by loading the CSV, confirming that the dashboard shows 30 records, entering different values and checking that a prediction and recommendations appear. The displayed accuracy is only a demonstration metric: with a very small synthetic dataset and one train/test split, it should not be interpreted as real-world model quality.

9. Benefits and Applications

The project is useful for learning the end-to-end data-science workflow. It can help demonstrate how data visualization, classification and user interfaces work together. A properly designed real-world version could support conversations between students and educators, but only with representative data, consent, privacy controls and human oversight.

10. Limitations and Future Enhancements

The dataset is small, synthetic and not representative. The model may be inaccurate or biased and cannot know every factor affecting a student's result. Future work could use a larger consented dataset, cross-validation, more evaluation metrics, secure data storage, explainable predictions, multilingual guidance and an approved generative-AI service for drafting study plans. No external GenAI API is connected in this demo.

11. Conclusion

The Student Success AI project demonstrates the practical use of Python libraries, data visualization and machine learning in an educational scenario. It provides an interactive example of prediction and study guidance while clearly communicating the limitations of the sample data and model.

References

Python documentation: https://docs.python.org/3/ ; Pandas documentation: https://pandas.pydata.org/docs/ ; Scikit-learn documentation: https://scikit-learn.org/stable/ ; Matplotlib documentation: https://matplotlib.org/stable/ ; Streamlit documentation: https://docs.streamlit.io/